

Dossiê de JavaScript Intermediário

Baseado no conteúdo do seu zip da Rocketseat, organizado para revisão rápida e prática.

Panorama rápido:

- Strings, arrays e objetos.
- Loops, datas, timezone e operador de coalescência.
- Funções construtoras, classes, herança, prototype chain e tratamento de erros.

1. Organização sugerida das pastas

A ideia aqui é separar por assunto, não por aula. Isso deixa a revisão muito mais rápida.

Pasta	Conteúdo
01-fundamentos	strings, operadores modernos e noções básicas
02-colecoes	arrays e objetos
03-controle-de-fluxo	for, while, do/while, break e continue
04-datas-e-horarios	Date, fuso horário e formatação
05-funcoes-e-poo	funções construtoras, classes, herança e prototype
06-erros	try/catch e erros personalizados

2. Strings

Você praticou maiúsculas/minúsculas, tamanho da string, replace, replaceAll, slice, trim, padStart, split, join, indexOf e includes.

```
// Manipulando texto em JavaScript
const texto = "Meus estudos de JavaScript"

console.log(texto.toUpperCase()) // MAIÚSCULO
console.log(texto.toLowerCase()) // minúsculo
console.log(texto.length) // quantidade de caracteres
console.log(texto.includes("JavaScript"))
console.log(texto.indexOf("estudos"))

const partes = texto.split(" ") // transforma em array
console.log(partes.join(" - ")) // junta novamente
```

```
// Cortando e ocultando dados
const cartao = "1234567812344928"
const ultimos4 = cartao.slice(-4)
const mascarado = ultimos4.padStart(cartao.length, "X")

console.log(ultimos4) // 4928
```

```
console.log(mascarado) // XXXXXXXXXXXX4928
```

3. Arrays

Você viu criação de array, acesso por índice, length, split, Array.from, includes, splice, slice, pop, shift e delete.

```
// Criando e lendo arrays
const alunos = ["Luke", "Gandalf", "Hermione", "Tony"]

console.log(alunos[0]) // primeiro item
console.log(alunos[alunos.length - 1]) // último item
console.log(alunos.includes("Tony"))

const letras = Array.from("Leo")
console.log(letras)
```

```
// Removendo e substituindo itens
const times = ["A", "B", "C", "D"]

times.splice(1, 1, "X") // remove B e coloca X
console.log(times)

const copia = times.slice(1, 3)
console.log(copia)

times.pop() // remove o último
times.shift() // remove o primeiro
```

4. Objetos

Você estudou propriedades, métodos, acesso por ponto e colchetes, atualização, delete, optional chaining e this.

```
// Objeto literal com método
const perfil = {
  nome: "Leonel",
  curso: "ADS",
  habilidades: ["HTML", "CSS", "JavaScript"],
  endereco: { cidade: "Tamboril" },
  apresentar() {
    console.log(`Olá, eu sou ${this.nome}`)
  }
}

console.log(perfil.habilidades[2])
console.log(perfil.endereco.cidade)
perfil.apresentar()
```

```
// Atualizando e acessando com segurança
perfil.periodo = 4 // adiciona
perfil.curso = "SI" // atualiza
delete perfil.aprovado // remove

const cidade = perfil?.endereco?.cidade
console.log(cidade)
```

5. Estruturas de repetição

Você praticou for, break, continue, for...of, for...in, while e do...while.

```
// for, break e continue
const nomes = ["Vitor", "Bruno", "Bruna"]

for (let i = 0; i < nomes.length; i++) {
  console.log(nomes[i])
}

for (let i = 0; i < 10; i++) {
  if (i === 5) break
}

for (let i = 0; i < 10; i++) {
  if (i === 5) continue
  console.log(i)
}
```

```
// while, do...while, for...of e for...in
let n = 1000
while (n > 10) {
  n /= 10
}

do {
  console.log("Executa pelo menos uma vez")
} while (false)

for (const nome of nomes) console.log(nome)

const aluno = { nome: "Leo", idade: 20 }
for (const chave in aluno) {
  console.log(chave, aluno[chave])
}
```

6. Datas e horários

Você viu criação de data, getTime, getDate, getMonth, getFullYear, getHours, getMinutes, setFullYear, setHours e formatação.

```
// Criando e lendo datas
const agora = new Date()
const dataEspecifica = new Date(2024, 6, 7, 5, 30, 0)

console.log(agora)
console.log(dataEspecifica)
console.log(dataEspecifica.getDay())
console.log(dataEspecifica.getDate())
console.log(dataEspecifica.getMonth())
console.log(dataEspecifica.getFullYear())
```

```
// Formatando
const data = new Date("2026-03-14T12:40:00")
const dia = String(data.getDate()).padStart(2, "0")
const mes = String(data.getMonth() + 1).padStart(2, "0")
```

```
const ano = data.getFullYear()

console.log(`${dia}/${mes}/${ano}`)
console.log(data.toLocaleDateString("pt-BR"))
console.log(data.toLocaleTimeString("pt-BR"))
```

7. Fuso horário

Você praticou timezone local, Intl.DateTimeFormat, getTimezoneOffset e data com offset.

```
// Fuso horário local
console.log(Intl.DateTimeFormat().resolvedOptions())
console.log(new Intl.DateTimeFormat("pt-BR").format(new Date()))

const agora = new Date()
console.log(agora.getTimezoneOffset()) // minutos
console.log(agora.getTimezoneOffset() / 60) // horas
```

```
// Criando data com offset
const dataComFuso = new Date("2024-12-05T22:56:30.603+03:00")
console.log(dataComFuso.toLocaleString())
```

8. Operador de coalescência nula

Você viu o ?? para usar um valor padrão quando a variável for null ou undefined.

```
// Valor padrão com ??
const conteudo = null
const resultado = conteudo ?? "conteúdo padrão"

console.log(resultado)
```

9. Função construtora

Você usou uma função como molde para gerar objetos com a mesma estrutura.

```
// Função construtora
function Product(nome) {
  this.nome = nome
  this.detalhes = function () {
    console.log(`O nome do produto é ${this.nome}`)
  }
}

const p1 = new Product("Teclado")
const p2 = new Product("Mouse")

p1.detalhes()
p2.detalhes()
```

10. Classes

Você praticou constructor, propriedades, métodos e método estático.

```
// Classe com método e método estático
class User {
  constructor(nome, email) {
    this.nome = nome
    this.email = email
  }

  sendEmail() {
    console.log(`Enviando e-mail para ${this.email}`)
  }

  static showMessage() {
    console.log("Mensagem estática")
  }
}

const user = new User("Leonel", "leo@email.com")
user.sendEmail()
User.showMessage()
```

11. Herança e sobrescrita

Você viu extends, reaproveitamento da classe base e sobrescrita de métodos.

```
// Herança e sobrescrita
class Animal {
  constructor(nome) {
    this.nome = nome
  }

  makeNoise() {
    console.log("Som genérico")
  }
}

class Dog extends Animal {
  makeNoise() {
    console.log("Woof, woof")
  }
}

const dog = new Dog("Chico")
dog.makeNoise()
```

12. Prototype chain

Você tocou no conceito de cadeia de protótipos com objetos e arrays.

```
// Objetos e arrays herdam comportamento via prototype chain
const address = { city: "Ceará", country: "Brasil" }
const users = ["Marcos", "Vilma", "Charles"]

console.log(address)
console.log(users)
// Métodos como toString, push e includes vêm da cadeia de protótipos
```

13. Tratamento de exceções

Você praticou try/catch, identificação de TypeError e erro personalizado.

```
// Capturando erros
try {
  const obj = []
  obj.execute()
} catch (error) {
  if (error instanceof TypeError) {
    console.log("Deu errado")
  }
}
```

```
// Erro customizado
class MyCustomError {
  constructor(message) {
    this.message = `CLASSE DE ERRO CUSTOMIZADA: ${message}`
  }
}

try {
  throw new MyCustomError("Erro personalizado lançado")
} catch (error) {
  console.log(error.message)
}
```

14. Como revisar esse material

A forma mais eficiente de revisar é por blocos: primeiro strings e arrays, depois objetos e loops, em seguida datas e OOP. Refaça os exemplos sem olhar, compare com o código comentado e tente explicar em voz alta o que cada linha faz.

Resumo final: o seu estudo já cobre a base intermediária boa de JavaScript. O próximo passo natural é consolidar DOM, funções mais avançadas, módulos, async/await e manipulação de APIs.

Material organizado para revisão prática.